

Search-R1：用强化学习训练大模型“边推理边自主调用搜索引擎”

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Ö. Arık,
Dong Wang, Hamed Zamani, Jiawei Han

University of Illinois at Urbana-Champaign (伊利诺伊大学厄巴纳-香槟分校)

University of Massachusetts Amherst (马萨诸塞大学阿默斯特分校)

Google Cloud AI Research (谷歌云 AI 研究院)

论文信息 会议/年份: COLM 2025 (arXiv:2503.09516) | 引用数: 1068 (Semantic Scholar, 2026-06-07, 本领域最高)

代码: <https://github.com/PeterGriffinJin/Search-R1>

一句话定位: 这篇论文要解决的问题是——怎么让大模型在自己思考的过程中，像人一样“想到不懂就去搜一下、读完再接着想、必要时再搜”，而且这种“边想边搜”的本事是被强化学习**训练出来的**，不是靠人写提示词硬凑出来的。它是“搜索类/深度研究类 (Deep-Research) 智能体用强化学习训练”这条技术路线的奠基之作。

摘要

摘要 (全文翻译) 对大语言模型 (large language models, LLMs) 来说，高效地获取外部知识与最新信息，对于有效的推理和文本生成至关重要。直接用提示词 (prompting) 让具备推理能力的先进 LLM 在推理 (inference) 时去使用搜索引擎，往往效果并不理想，因为模型可能并不真正掌握“如何与搜索引擎进行最优交互”的能力。本文提出 Search-R1，它是对“面向推理的强化学习 (reinforcement learning, RL) 框架”的一种扩展：让 LLM 学会在逐步推理的过程中，配合实时检索，自主地生成 (多次) 搜索查询。Search-R1 通过带多轮搜索交互的方式来优化 LLM 的推理轨迹 (trajectory)，并借助对检索回来的 *token* 做损失掩码 (*retrieved token masking*) 来实现稳定的 RL 训练，同时采用一个简单的、基于最终结果的奖励函数 (outcome-based reward)。在 7 个问答 (question-answering) 数据集上的实验表明，在相同实验设置下，Search-R1 相对各类 RAG 基线分别提升了 24% (Qwen2.5-7B) 和 20% (Qwen2.5-3B)。本文还就 RL 优化方法、LLM 模型选择，以及检索增强推理中“回复长度的动态变化”给出了实证洞察。代码与模型检查点 (checkpoints) 已开源：<https://github.com/PeterGriffinJin/Search-R1>。

1 小白知识卡：读这篇前要懂的几个词

下面这些概念在本文反复出现，先用大白话过一遍，后面读正文就轻松了。

- **大语言模型 (LLM)**: 就是会“接话”的模型（如 GPT、Qwen）。它根据已经看到的文字，一个 token 一个 token 地往下预测、生成内容。
- **RAG (检索增强生成, retrieval-augmented generation)**: 传统做法——先拿用户问题去知识库里“搜一把”，把搜到的几段文字塞进模型的输入里，模型再据此作答。缺点是只搜一次、而且模型本身并没有被训练“怎么搜更聪明”。
- **把搜索当工具 (search-as-a-tool)**: 让模型在思考过程中主动发起搜索（像 ReAct、IRCoT 那样）。但若只靠写提示词，模型常常用不好；若靠监督微调 (SFT)，又得人工标注大量“正确的搜索轨迹”，成本极高。
- **强化学习 (RL)**: 一种“试错 + 给分”的训练范式。模型（这里叫策略 *policy*）去做一连串决策，环境给它一个奖励 (reward) 分数，模型据此调整自己、争取拿更高分。它最大的好处是：只告诉模型“最终对没对”，模型自己就能摸索出复杂的中间策略。
- **PPO (近端策略优化, proximal policy optimization)**: 最常用的一种 RL 算法。它需要额外训练一个“裁判”（叫价值模型/*critic*）来估计“当前局面值多少分”，训练更稳但更重。
- **GRPO (组相对策略优化, group relative policy optimization)**: DeepSeek 提出的简化版 RL。它不另外训裁判，而是对同一个问题采样一组答案，用这组答案的平均分当基准，谁高于平均就鼓励、谁低于平均就抑制。更轻、收敛更快，但有时会“崩”。
- **RLVR (带可验证奖励的强化学习, RL with verifiable rewards)**: 奖励不靠另一个神经网络打分，而是用规则直接判对错（比如答案字符串是否精确匹配标准答案）。本文用的就是这种，简单又不容易被模型“钻空子”。
- **DeepSeek-R1 / R1-Zero**: 用纯 RL（只给结果奖励）就让模型自己学会长链推理、自我验证、自我纠错的代表作。本文可以理解成它的“加搜索版”：R1 让模型“在脑子里想”，Search-R1 让模型“想不通就上网查”。
- **token 掩码 (token masking)**: 训练时把某些 token 从“算账”里排除。本文的关键技巧——搜索引擎返回的那段文字不是模型自己写的，所以训练时不让模型为这段内容的概率“负责/背锅”，否则训练会乱套。

2 引言在讲什么（对应原文第 1 节）

背景痛点。 LLM 很能“说”，但有两个老毛病：(1) 做复杂推理容易出错；(2) 缺乏最新的、领域专门的外部知识，还爱“一本正经地胡说”（即幻觉，hallucination）。要治这两个病，就得让模型既会推理、又会高效地用搜索引擎调外部信息。

现有两条路都不够好。 论文把已有做法分成两类：

- **RAG**: 基于输入去检索段落、拼进上下文再生成。哪怕有人改成“多轮多次检索”，模型也没在训练中学过怎么和搜索引擎打交道，所以是次优的。
- 把搜索当工具：用提示词驱动（泛化差，没见过的任务就抓瞎）或用监督微调（要海量高质量标注轨迹，难以规模化）。更要命的是——搜索这个动作本身是“不可微”的（你没法对“去网上搜一下”这个离散操作求梯度），所以端到端的梯度下降训不动它。

RL 是突破口，但有三道坎。 既然 OpenAI-o1、DeepSeek-R1 证明了“只给结果奖励、用 RL”就能逼出强推理能力，那能不能把搜索也用 RL 训出来？作者指出三个核心挑战：

1. **RL 框架与稳定性**：怎么把搜索引擎塞进 RL 流程，尤其是检索回来的上下文混进序列后，怎么保证训练稳定？
2. **多轮交错的“推理-搜索”**：理想情况下，模型应能反复地“想—搜—再想—再搜”，并根据问题难度动态调整检索策略。
3. **奖励设计**：到底用不用复杂的“过程奖励”？还是说一个简单的“只看最终答案对错”的奖励就够了？

本文的答案——Search-R1 的三个关键创新：

1. 把搜索引擎建模为环境的一部分，于是采样出的轨迹就是“模型生成的 token”和“搜索返回的 token”交错在一起的序列。框架同时兼容 PPO 与 GRPO，并用**检索 token 掩码**保证训练稳定。
2. 支持多轮检索与推理：用 `<search>...</search>` 触发搜索，检索结果包在 `<information>...</information>` 里，推理过程包在 `<think>...</think>` 里，最终答案放在 `<answer>...</answer>` 里，形成结构化的迭代决策。
3. 只用极简的“结果奖励”（答对就给分），不搞复杂的过程奖励。实验证明这个最小化设计就够用。

可以把 Search-R1 看成 DeepSeek-R1-Zero 的扩展：后者主攻“用模型自身参数里的知识做推理”，前者引入“搜索增强的 RL 训练”，让决策能由外部检索来驱动。

3 相关工作速览（对应原文第 2 节）

LLM 与检索。 给 LLM 接外部信息主要两条路：RAG（检索一次 → 顺序生成，问题是可能检到无关内容、上下文不够用）和把搜索当工具（IRCoT、ReAct 用提示词驱动迭代推理 + 搜索；Toolformer 用监督微调增强用工具的能力，但依赖难以规模化的高质量标注轨迹）。最近的工作（DeepSeek-R1）显示：只靠结果奖励，RL 就能逼出高级推理，但它在“调用搜索引擎”这个场景下还没被充分探索——这正是本文的切入点。

LLM 与强化学习。 RL 进入 LLM 调优是从 RLHF（基于人类反馈的强化学习）开始的：先用人类偏好数据训一个奖励模型，再用它（通常是 PPO）去调策略 LLM。但 PPO 工程上较重。于是出

现了 DPO（直接偏好优化）、SimPO 等直接优化方法，省事但有“离策略（off-policy）”问题，性能不总能赶上纯 RL。另一支是 GRPO（去掉 critic，用一组样本的平均分当基准）和 RLOO（简化版 REINFORCE）。但把 RL 用到“LLM 驱动搜索交互 + 推理”上，此前几乎是空白。

4 方法：Search-R1 怎么做（对应原文第 3 节）

这一节是全文核心，分四块讲：（1）把搜索引擎接进 RL 的目标函数；（2）多轮搜索的生成流程；（3）训练用的模板；（4）奖励设计。

4.1 3.1 把搜索引擎接进强化学习

优化目标（公式 1）。 作者把“带搜索引擎 R ”的 RL 目标写成：

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(\cdot | x; R)} [r_{\phi}(x, y)] - \beta D_{\text{KL}}[\pi_{\theta}(y | x; R) \parallel \pi_{\text{ref}}(y | x; R)]. \quad (1)$$

逐符号拆解：

- π_{θ} ：要训练的策略模型（就是我们的 LLM）； π_{ref} ：参考模型（训练前的初始模型，当“锚点”用，防止训歪）。
- $x \sim D$ ：从数据集 D 里抽一个输入（问题）； $r_{\phi}(x, y)$ ：奖励函数，给“问题 x + 回答 y ”打分。
- $y \sim \pi_{\theta}(\cdot | x; R)$ ：注意这里的 y 不再是模型一口气写完的，而是“推理 token”和“搜索引擎检索结果”交错组成的序列——这就是和普通 RL 最大的区别。作者把它记作 $\pi_{\theta}(\cdot | x) \otimes R$ ， $\otimes$ 表示“检索与推理交错进行”。
- $D_{\text{KL}}[\cdot \parallel \cdot]$ ：KL 散度，衡量新策略和参考模型差多远； β 是它的权重。这一项是“别跑太远”的缰绳，保证训练稳定。

直觉：在“答对就给分”的驱动下，模型自己去摸索“什么时候该搜、搜什么、怎么用搜到的东西”，同时别离初始模型太远以免崩。框架建立在两大经典策略梯度方法 PPO 和 GRPO 之上。

关键技巧：对检索 token 做损失掩码（Loss Masking for Retrieved Tokens）。 这是本文最重要的工程洞见，务必理解：

在 PPO/GRPO 里，token 级的损失是对整条轨迹逐 token 计算的。而 Search-R1 的轨迹里，既有“模型自己生成的 token”，也有“从外部段落检索回来的 token”。**优化模型自己写的 token**（怎么发起搜索、怎么推理）是我们想要的；但如果把同样的优化也套到检索回来的 token 上，就会导致“意料之外的学习动态”——说白了，搜索引擎返回什么内容不是模型能控制的，硬要模型去“拟合/对这段内容负责”就会把训练带乱。

解决办法：引入一个掩码 $I(y_t)$ ——若 y_t 是模型生成的 token 则 $I(y_t) = 1$ ，若是检索回来的 token 则 $I(y_t) = 0$ ；策略梯度只在模型生成的 token 上计算。这既稳定了训练，又保留了“搜索增强生成”的灵活性。**这是 Search-R1 能稳定训练的命门。**

PPO 版（公式 2）。 PPO 是常用的 actor-critic 方法，其（带搜索的）目标为：

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim D, y \sim \pi_{\text{old}}} \left[\frac{1}{\sum_{t=1}^{|y|} I(y_t)} \sum_{t=1: I(y_t)=1}^{|y|} \min \left(\frac{\pi_{\theta}(y_t|x, y_{<t}; R)}{\pi_{\text{old}}(y_t|x, y_{<t}; R)} A_t, \text{clip} \left(\frac{\pi_{\theta}(y_t|x, y_{<t}; R)}{\pi_{\text{old}}(y_t|x, y_{<t}; R)}, 1-\epsilon, 1+\epsilon \right) A_t \right) \right]. \quad (2)$$

怎么读： π_{θ} 是当前策略、 π_{old} 是上一步的旧策略，二者之比是重要性采样比（衡量“同一个 token 在旧策略下概率变化多少”）。 A_t 是优势（*advantage*），表示“这一步比平均水平好多少”，由 GAE（广义优势估计）基于未来奖励和一个学到的价值函数 V_{ϕ} 算出。 $\text{clip}(\cdot, 1-\epsilon, 1+\epsilon)$ 把这个比值裁剪在 $[1-\epsilon, 1+\epsilon]$ ，防止单步更新过猛——这正是 PPO 稳定的来源。注意求和写的是 $t = 1 : I(y_t) = 1$ 、分母是 $\sum I(y_t)$ ：**只对“模型自己写的 token”求和、求平均，检索 token 被掩掉了。**

GRPO 版（公式 3）。 为了不用额外的价值函数、提升稳定性，GRPO 对每个问题 x 采样一组回答 $\{y_1, \dots, y_G\}$ ，用组内相对奖励来算优势 $\hat{A}_{i,t}$ （谁比这一组的平均好就鼓励）：

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{\sum_t I(y_{i,t})} \sum_{t: I(y_{i,t})=1} \left(\min \left(\frac{\pi_{\theta}(y_{i,t}|\cdot)}{\pi_{\text{old}}(y_{i,t}|\cdot)} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(y_{i,t}|\cdot)}{\pi_{\text{old}}(y_{i,t}|\cdot)}, 1-\epsilon, 1+\epsilon \right) \hat{A}_{i,t} \right) - \beta D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right) \right]. \quad (3)$$

和 PPO 的两点不同：（1）优势 $\hat{A}_{i,t}$ 用组内相对奖励算，不需要 critic；（2）KL 惩罚不是塞进奖励里，而是直接加在损失上。同样地，算 KL 损失时**也对检索 token 做掩码**。

4.2 3.2 多轮搜索引擎调用的生成流程

模型的回答不是一次成型，而是在“文本生成”和“查搜索引擎”之间反复切换。系统提示告诉模型：需要检索时，把查询写在 `<search>...</search>` 之间；系统一旦在生成里检测到这对标签，就提取查询、去搜、把结果包在 `<information>...</information>` 里追加到序列后面，作为下一步生成的额外上下文。如此往复，直到：（1）达到最大动作次数，或（2）模型给出最终答案（包在 `<answer>...</answer>` 里）。完整流程见原文算法 1（Algorithm 1），核心循环用大白话说就是：

算法 1（多轮搜索的回复生成，大白话版）： 给定问题 x 、策略模型、搜索引擎、最大动作预算 B 。

重复至多 B 次：让模型往下生成 token，直到吐出 `</search>`、`</answer>` 或结束符就停；如果这一

段里出现了 `<search>...</search>`，就把查询抠出来、调搜索引擎拿结果 d ，并以 `<information>d</information>` 的形式插回序列；如果出现了 `<answer>...</answer>`，就返回最终答案；否则（动作不合法）追

加一句“My action is not correct. Let me rethink.”逼模型重想。

4.3 3.3 训练模板（对应原文表 1）

为了引导初始模型按格式输出，作者用了一个极简模板，把输出结构化成三部分循环：先推理，再发起搜索，最后给答案。**关键是：模板只规定“格式”，不规定“内容”**——不强迫模型必须反思、必须搜索、也不偏向某种解题套路，这样才能原汁原味地观察 RL 过程中模型自发的学习动态。模板原文（保留英文，便于复现）大意是：

回答给定的问题。每次拿到新信息，你必须先在 `<think>` 和 `</think>` 之间进行推理。推理之后，若发现自己缺少某些知识，可以用 `<search>` 查询 `</search>` 来调用搜索引擎，它会把检索到的 `top` 结果返回在 `<information>` 和 `</information>` 之间。你想搜多少次都行。若发现不再需要外部知识，就可以直接把答案写在 `<answer>` 和 `</answer>` 之间，不必详细解释。例如 `<answer>xxx </answer>`。问题：{question}。

4.4 3.4 奖励设计（公式 4）

奖励是 RL 的主要训练信号。Search-R1 采用纯基于规则、只看最终结果的奖励系统，用来判断模型答得对不对。例如事实类推理任务，可以用精确字符串匹配（exact match, EM）：

$$r_{\phi}(x, y) = \text{EM}(a_{\text{pred}}, a_{\text{gold}}), \quad (4)$$

其中 a_{pred} 是从回答 y 里抽出的最终答案， a_{gold} 是标准答案。两点设计取舍值得注意：（1）**不加格式奖励**——因为训出来的模型本身就很守格式（这点和 DeepSeek-R1 不同，R1 用了格式奖励）；（2）**不训神经网络奖励模型**——因为大规模 RL 下 LLM 对奖励形式很敏感、易被“钻空子”，而且再训一个奖励模型又费算力又复杂。一句话：奖励越简单、越“可验证”，越稳。

5 方法图解：一张图看懂训练流程

图 1 是全文的“总览图”，展示了 Search-R1 分别用 PPO（上半部分）和 GRPO（下半部分）训练时，数据是怎么流动的。

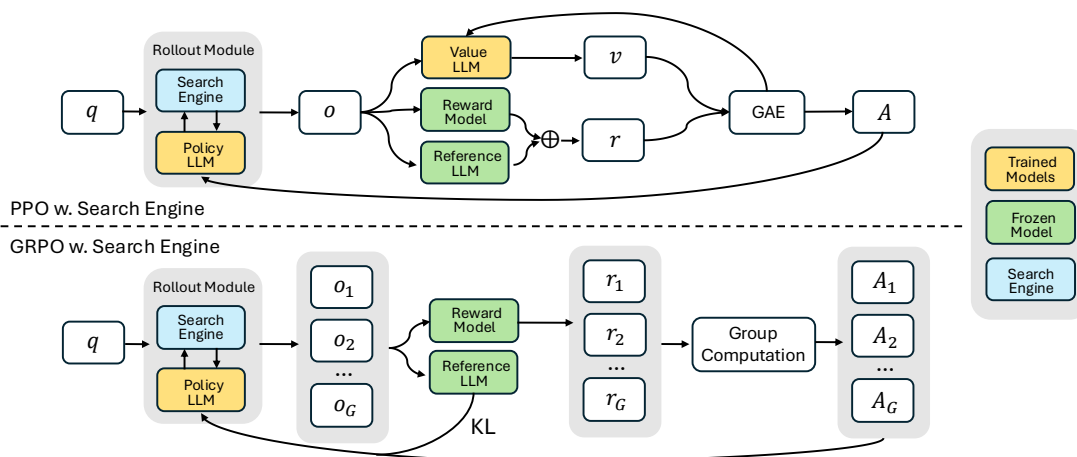


Figure 1: Demonstration of PPO and GRPO training with the search engine (SEARCH-R1). During the rollout, LLMs can conduct multi-turn interactions with the search engine.

图 1: 用搜索引擎进行 PPO 与 GRPO 训练的示意图 (Search-R1)。在 rollout (轨迹采样) 阶段, LLM 可以与搜索引擎进行多轮交互。图例: 黄色 = 被训练的模型, 绿色 = 冻结不动的模型, 蓝色 = 搜索引擎。

图里在说什么、数据怎么流动？

- **Rollout Module (轨迹采样模块, 左侧灰框)**: 这是和别的 RL 框架最不一样的地方。问题 q 进来后, *Policy LLM* (策略模型, 黄色, 要训练) 和 *Search Engine* (搜索引擎, 蓝色) 反复来回交互——模型生成查询 → 搜索引擎返回结果 → 模型接着生成……最终产出一条“推理与检索交错”的完整轨迹 o (GRPO 下是一组 $o_1, \dots, o_G$)。
- **上半部分 (PPO)**: 轨迹 o 分别送给三个模型——*Value LLM* (价值模型, 黄色, 要训练) 输出价值 v ; *Reward Model* (奖励模型) 给出奖励 r ; *Reference LLM* (参考模型, 绿色, 冻结) 用来算 KL (图中 $\oplus$ 表示把 KL 惩罚并入)。然后 v 和 r 一起喂给 **GAE** 算出优势 A , 再用 A 去更新策略。注意这里需要一个额外的 Value LLM, 所以 PPO “更重”。
- **下半部分 (GRPO)**: 对同一个 q 采样一组轨迹 $o_1, \dots, o_G$, 每条各自拿到奖励 $r_1, \dots, r_G$, 送进 **Group Computation (组内计算)**, 用“组内相对好坏”直接算出各自的优势 $A_1, \dots, A_G$ 。没有 **Value LLM**、没有 **GAE**, 所以 GRPO “更轻、收敛更快”, 但也更容易不稳定 (后面实验会看到它训久了可能“崩”)。Reference LLM 仍用于算 KL。

一句话总结这张图: 两条路线的唯一公共关键就是左边那个“模型 搜索引擎来回交互”的 *Rollout Module*; 右边只是 PPO 和 GRPO 各自不同的“怎么把奖励折算成更新信号”的算法差异。

6 实验解读（对应原文第 4 节）

6.1 用了哪些数据集、对比哪些基线

7 个问答数据集（涵盖单跳与多跳，既测“训练域内”也测“域外泛化”）：

- 通用问答 (*General QA*)：NQ (Natural Questions)、TriviaQA、PopQA。
- 多跳问答 (*Multi-Hop QA*, 需要分步推理、串联多条线索)：HotpotQA、2WikiMultiHopQA、Musique、Bamboogle。

其中 **NQ 和 HotpotQA 是“训练域内”** (in-domain, 记 $\dagger$, 训练数据就是这俩合并而成), 其余 5 个是 **“训练域外”** (out-of-domain, 记 $\star$, 从没在训练里见过, 用来检验泛化)。

对比的基线 (baselines) 三大类：(1) 不检索：直接推理 (Direct Inference)、思维链 (Chain-of-Thought, CoT); (2) 带检索：RAG、IRCoT、Search-o1; (3) 微调类：SFT (监督微调)、R1 (用同样数据做 RL 但不带搜索)、带搜索的拒绝采样 (rejection sampling, 挑出答对的轨迹再去微调)。

公平性设置：所有方法用同一个检索器 (E5)、同一个知识库 (2018 年维基百科)、同样检索 3 段 (top-3)、同样的训练数据和同样的基座模型 (Qwen2.5-3B/7B 的 Base/Instruct 版)。评测指标用 EM (精确匹配)。默认 RL 算法是 PPO。

6.2 主结果：Search-R1 全面领先

表 1 是核心结果（节选 7B 与 3B 的关键行，完整数据见原文表 2）。

表 1：主结果（节选）。数值为 EM（越高越好）， $\dagger$ = 训练域内、 $\star$ = 训练域外；**加粗**为该规模下最佳。Avg.= 七数据集平均。

方法	NQ $\dagger$	TriviaQA $\star$	PopQA $\star$	HotpotQA $\dagger$	2wiki $\star$	Musique $\star$	Bamboogle $\star$	Avg.
<i>Qwen2.5-7B</i>								
Direct Inference	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.181
RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.304
R1-base (无搜索 RL)	0.297	0.539	0.202	0.242	0.273	0.083	0.296	0.276
Rejection Sampling	0.360	0.592	0.380	0.331	0.296	0.123	0.355	0.348
Search-R1-base	0.480	0.638	0.457	0.433	0.382	0.196	0.432	0.431
Search-R1-instruct	0.393	0.610	0.397	0.370	0.414	0.146	0.368	0.385
<i>Qwen2.5-3B</i>								
RAG	0.348	0.544	0.387	0.255	0.226	0.047	0.080	0.270
R1-base (无搜索 RL)	0.226	0.455	0.173	0.201	0.268	0.055	0.224	0.229
Search-R1-base	0.406	0.587	0.435	0.284	0.273	0.049	0.088	0.303
Search-R1-instruct	0.341	0.545	0.378	0.324	0.319	0.103	0.264	0.325

这些数字说明了什么（原文四点结论）：

1. **Search-R1 稳定碾压强基线**：相对各类 RAG 基线，7B 平均提升 24%、3B 提升 20%（论文摘要口径；按“两个模型平均相对提升”另一口径为 41% 和 20%）。无论域内（NQ/HotpotQA）还是域外（其余 5 个）都成立——说明它学到的是通用的“会搜会想”能力，而非死记某几个数据集。
2. **比“无搜索的 RL (R1)”更强**：把搜索引入推理，等于给模型开了“查资料”的外挂，自然更准。这也反过来说明：光靠模型脑子里的知识不够，接外部检索是真有用。
3. **Base 和 Instruct 模型都吃这一套**：证明 DeepSeek-R1-Zero 那种“只用结果奖励的 RL”不仅能练纯推理，也能成功迁移到“推理 + 搜索”场景。
4. **模型越大越会搜**：7B 相对第二名（RAG）的“性能差距”明显大于 3B——大模型更能学会“如何把搜索用好”。

6.3 消融与分析（对应原文第 5 节，几条最有用的）

（1）PPO vs. GRPO（原文图 2a、表 3）：

- GRPO 收敛更快（因为 PPO 要先花几步“热身”那个 critic）；
- 但 PPO 更稳——GRPO 训久了会出现“奖励崩塌（reward collapse）”，PPO 始终平稳；
- 两者最终性能相当。所以本文默认用 PPO（求稳）。

（2）**检索 token 掩码到底有多重要**（原文图 3、表 4）：这是验证那个“命门技巧”的关键消融。结果非常干脆——加掩码全面优于不加：

表 2：检索 token 损失掩码的消融（Qwen2.5-7B-base, PPO）。加掩码在全部 7 个数据集上一致更好。

方法	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	Avg.
Search-R1 加掩码	0.480	0.638	0.457	0.433	0.382	0.196	0.432	0.431
Search-R1 不加掩码	0.388	0.567	0.391	0.325	0.321	0.108	0.304	0.343

平均分从 0.343 跳到 0.431——“**不让模型为搜索引擎返回的内容背锅**”这一招，直接带来约 0.09 的平均提升，是 Search-R1 能稳定见效的核心原因之一。

（3）回复长度与搜索次数的动态（原文图 2c、2d）：训练过程呈现有趣的“先降后升再稳”规律

- 前 100 步：回复长度骤降，奖励微升。此阶段模型在“删废话”，去掉冗余的填充词、适应任务。
- 100 步以后：回复长度和奖励一起显著上升。此时模型学会频繁调用搜索引擎，检索段落让回复变长，而善用搜索结果让奖励大涨。

- 与之对应，有效搜索次数随训练持续增多——模型是“自发”学会了多搜、反复搜的，没人教它。

(4) **Base vs. Instruct** (原文图 2b): Instruct 模型起点高、收敛快，但最终两者性能很接近——说明 RL 有能力把差距补平，让 Base 模型也达到相当水平。

(5) **其它 (附录)**: 在 14B 模型上 Search-R1 依然全面领先且随规模继续涨 (附录 C); 检索段数 $\text{top-}k=3$ 综合最优 (太少召回不够、太多引入噪声反而拖累, 附录 G); GRPO 的组大小 (group size) 越大收敛越快但越易崩, 小组更稳、泛化更好 (附录 H)。

6.4 案例研究：模型“边想边搜、还会自我验证”

原文附录用大量案例展示了训练后模型的行为。最经典的一例 (对比 R1 与 Search-R1, 原文表 9):

问题: Curious 这款女士香水的代言歌手，出生在哪个城市和州? (标准答案: McComb, Mississippi)

R1 (无搜索，只靠脑补): <think> 这位歌手是 Beyoncé, 出生在休斯顿……</think> <answer> Houston </answer> ——答错了 (连歌手都认错)。

Search-R1 (边搜边想): 先搜 “Curious fragrance information” → 得知是 Britney Spears 代言; 再搜 “Britney Spears birthplace” → 得知出生地 McComb, Mississippi; 此时其实已经够了, 但模型又多搜了一次 “McComb, Mississippi location” 来自我验证, 最后 <answer> McComb, Mississippi </answer> ——答对, 且过程严谨。

作者从案例里总结出两点: (1) **交错的推理与检索能加深问题分析**——模型可以一边搜一边迭代地修正自己的推理; (2) **通过反复检索做自我验证 (self-verification)**——哪怕信息已足够, 模型仍会再搜一次确认, 这和“无搜索的推理 RL (DeepSeek-R1)”里观察到的“验证驱动的推理”一脉相承。当然, 附录也诚实地给了失败案例: 复杂问题分解不当、或被无关检索结果误导时, 模型仍会答错 (如把多跳问题想偏、被噪声段落带跑)。

7 核心 takeaways 与对 AI 应用开发的启示

这篇论文最值得记住的几点:

1. **“会搜”可以被训练，而不是靠提示词硬凑**。把搜索引擎当成环境的一部分，用 RL 端到端地训，模型就能自主学会“何时搜、搜什么、怎么用”。这是从“RAG/工具调用靠人设计”到“搜索行为靠模型自学”的范式转变。
2. **最关键的工程技巧是“检索 token 掩码”**: 训练时不让模型为搜索引擎返回的文字负责，否则训练会崩。这一招让平均分从 0.343 提到 0.431。凡是“模型序列里混入了非模型生成的外部内容”的 RL 场景，都该想到这一点。

3. 简单的“结果奖励 (RLVR)”就够用：只用规则判答案对错 (EM)，不搞复杂过程奖励、不训神经奖励模型——既省事又稳、还不易被钻空子。
4. 多轮交错 (推理 检索) + 自发涌现的行为：模型在 RL 中自己学会了多次检索、反复验证、纠错——这些没有被显式教过，是奖励信号逼出来的。
5. 算法选型有取舍：GRPO 快但易崩、PPO 稳但慢，最终性能相当，求稳选 PPO；检索段数、组大小都存在“质量 vs. 稳定”的权衡。

对 AI 应用开发实习生的实际启示：

- 做检索类/问答类 *Agent*，别只停在“RAG 检一次”。如果你的场景是多跳、需要分步查资料（如客服、研究助理、Deep-Research 类产品），Search-R1 这种“多轮自主检索”的形态是更强的目标形态。
- 如果你要用 RL 训这类 *Agent*，务必对“工具/检索返回的内容做掩码”。这是从踩坑里换来的稳定性经验，能省掉大量“训练莫名崩掉”的调试时间。
- 奖励能用规则就别用模型。在答案可自动判对错的任务上（数学、事实问答、代码能跑通），优先用可验证奖励，既便宜又稳。
- 先用结构化标签把“想/搜/答”框起来（`<think>/<search>/<information>/<answer>`），系统层据此解析并接管搜索调用——这套“标签协议 + 系统接管工具执行”的工程范式，即使你暂时不做 RL、只做提示工程，也很值得直接借鉴。
- 基座可以从 *Base* 模型起步。RL 能把 *Base* 和 *Instruct* 的差距补平，不一定非得用昂贵的指令模型当起点。

参考文献（仅列最关键几条，英文原样）

1. Daya Guo, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.
2. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
3. Zhihong Shao, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models (GRPO). arXiv preprint arXiv:2402.03300, 2024.
4. Patrick Lewis, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks (RAG). NeurIPS, 2020.
5. Harsh Trivedi, et al. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions (IRCoT). arXiv:2212.10509, 2022.
6. Shunyu Yao, et al. ReAct: Synergizing reasoning and acting in language models. ICLR, 2023.

7. Timo Schick, et al. Toolformer: Language models can teach themselves to use tools. NeurIPS, 2023.
8. Xiaoxi Li, et al. Search-o1: Agentic search-enhanced large reasoning models. arXiv:2501.05366, 2025.